Azure Landing Zone with Terraform

Implementation Solution Guide

Version 1.0.0 · 2026-07-09

Contents

Executive Summary — Azure Landing Zone with Terraform 5

The problem . . . 5

The solution . . . 5

What's included . . . 5

Business value . . . 6

Effort and boundary . . . 6

1. Solution Overview — Azure Landing Zone with Terraform 6

What this solution is . . . 6

How the technology works . . . 6

What this solution builds (the components) . . . 7

The standard deployment model . . . 7

What is out of scope . . . 7

Authentication and connections used . . . 8

Limitations (be honest with buyers) . . . 8

2. Requirements Guide — Azure Landing Zone with Terraform 8

2.1 Elevated rights at the tenant-root / management-group scope . . . 8

2.2 Terraform + provider versions pinned (and available) . . . 9

2.3 Remote state backend provisioned (storage account + container + locking) . . . 9

2.4 CI/CD service connection using workload identity federation (OIDC) . . . 9

2.5 Naming, tagging, and allowed-region standard defined . . . 9

2.6 A non-production target scope to validate against . . . 10

2.7 Microsoft Graph / Az PowerShell for the helper scripts . . . 10

Checklist . . . 10

Global Requirements — DevOps (category-level) 10

G1. Target cloud subscription/tenant with sufficient rights . . . 11

G2. IaC tooling installed and versioned . . . 11

G3. Source control repository . . . 11

G4. CI/CD platform and service connection . . . 11

G5. Remote state / deployment backend . . . 11

G6. Secrets management . . . 11

G7. Naming, tagging, and region standards . . . 12

G8. Change-control and non-production target . . . 12

3. Implementation Guide — Azure Landing Zone with Terraform 12

Step 1 — Assess: baseline the current estate . . . 13

Step 2 — Bootstrap the remote state backend (one-time, outside the pipeline) . . . 13

Step 3 — Configure: author the Terraform and pin versions . . . 13
Step 4 — Pilot: plan and apply against the non-production scope . . . 14
Step 5 — Wire the CI/CD pipeline (plan → approve → apply) . . . 14
Step 6 — Validate the pilot (the gates) . . . 14
Step 7 — Expand: promote to production in waves . . . 15
5. Testing & Validation — Azure Landing Zone with Terraform 15
Test matrix . . . 15
Automated checks . . . 16
Evidence to capture . . . 17
Exit criteria (before Expand / before `Deny`) . . . 17
6. Troubleshooting — Azure Landing Zone with Terraform 17
`terraform init` fails on the backend . . . 17
"Error acquiring the state lock" / apply hangs on lock . . . 17
`plan`/`apply` fails with authorization / "does not have permission" at MG scope . . . 17
Provider/plugin version errors or an unexpected plan after `init -upgrade` . . . 18
Policy assignment is blocking a legitimate deployment (`RequestDisallowedByPolicy`) . . . 18
Policy shows "not started" / no compliance results . . . 18
Management group changes don't appear / hierarchy looks wrong . . . 18
Pipeline can't authenticate to Azure (OIDC) . . . 19
The `apply` stage ran without approval . . . 19
VNet peering fails . . . 19
Scheduled `plan` reports drift you didn't make . . . 19
7. Best Practices — Azure Landing Zone with Terraform 20
Plan-first, always . . . 20
State discipline . . . 20
Guardrails (policy) as code . . . 20
Hierarchy & subscription vending . . . 20
Networking . . . 21
Pipeline & drift . . . 21
Version safety . . . 21
Change safety . . . 21
Hygiene . . . 21
Where to go next (adjacent catalog areas) . . . 22
8. Deliverables Summary — Azure Landing Zone with Terraform 22
What you receive . . . 22
Outcomes . . . 23

Assumptions & boundaries 23
Appendix A — PowerShell Scripts 25

Executive Summary — Azure Landing Zone with Terraform

The problem

Most Azure estates start the same way: someone creates a subscription, clicks resources into the portal, and the organization is live. It doesn't scale. There is no management-group hierarchy to inherit policy, no guardrails to stop resources landing in the wrong region or without tags, no consistent RBAC, and no record of who changed what. Within months the environment has **drifted** — the portal no longer matches anyone's intent — and every new subscription repeats the manual work. Governance, cost control, and security all suffer because the foundation was never codified.

The solution

This solution deploys an **Azure landing zone as code with Terraform** — a governed, versioned cloud foundation you stamp out repeatably:

1. **Management group hierarchy + subscription vending** — a Cloud Adoption Framework-aligned hierarchy

(Platform / Landing Zones / Sandbox / Decommissioned) so policy and RBAC **inherit** downward, and a repeatable pattern for placing new subscriptions.

1. **Azure Policy guardrails as code** — allowed regions, required tags, and security baselines assigned

at the right scope in **audit/**`Disabled` **first**, then enforced (`Deny`) after validation.

1. **Identity & RBAC baseline + remote Terraform state** — least-privilege role assignments defined in

code, with state stored in a locked, versioned Azure Storage backend (never on a laptop).

1. **Hub-spoke networking & DNS** — a connectivity hub with spokes, peering, and private DNS, ready for

workloads.

1. **CI/CD pipeline (plan → approve → apply) with drift detection** — every change runs `terraform plan`

for review, requires a manual approval before `apply`, and a scheduled plan flags drift.

Delivery follows a disciplined **Assess → Configure → Pilot → Validate → Expand** model: discover the current estate, author the modules, run **plan/what-if** against a **non-production** target first, prove the gates, then promote to production behind an approval gate — never an unreviewed apply against prod.

What's included

Full documentation (overview, requirements, ordered implementation, testing matrix, troubleshooting, best practices) plus **five deployable artifacts**: a read-only **discovery** script, the core Terraform (`main.tf`), a **validation** script (plan + policy-compliance), a **CI/CD pipeline** (`azure- pipelines.yml`) with plan/apply approval and drift detection, and a **drift/governance report** script. Everything is **plan/what-if-first and approval-gated**; no secrets live in code.

Business value

- **Repeatable & governed** — stamp out subscriptions from the same code, with policy and RBAC inherited.
- **No drift** — state + a scheduled plan catch out-of-band changes before they compound.
- **Auditable** — every change is a reviewed pull request and an approved pipeline run.
- **Safe** — plan/what-if against non-prod, manual approval before any prod apply, and a clear rollback.

Effort and boundary

A competent admin comfortable with Terraform and Azure can stand up the pilot landing zone (hierarchy, state, baseline policy, one hub) in a few focused sessions, then expand. This guide provides the **technical foundation and the automation**. Your organization's **naming/tagging standard, allowed regions, RBAC model, and production change-approval process** are governance decisions the guide encodes but does not decide for you.

> ***Version note:*** *Terraform, the AzureRM/AzAPI providers, and any Azure landing-zone modules are* ***version-sensitive****. All provider and module versions in this solution are* ***pinned and flagged "verify"*** *— confirm current supported versions against the provider/module registries before you apply. *Verify current availability and version compatibility for your tenant.**

1. Solution Overview — Azure Landing Zone with Terraform

What this solution is

An **Azure landing zone** is a pre-provisioned, governed environment — management groups, policy, RBAC, networking, and logging — that workloads land into, rather than each team building a foundation from scratch. This solution builds that landing zone **as code with Terraform**, so the whole foundation is **versioned in Git, reviewed as pull requests, and deployed through a pipeline** with plan-before-apply and drift detection. It is aligned to the Microsoft **Cloud Adoption Framework (CAF)** enterprise-scale pattern, kept deliberately lean so a competent admin can own it.

How the technology works

- **Terraform** describes the desired state of Azure resources in `.tf` files. `terraform plan` shows

exactly what will change (**what-if**); `terraform apply` makes it so. Because the state is stored remotely and the code is in Git, the environment is reproducible and drift is detectable.

- **Providers** — the **AzureRM** provider manages most Azure resources; **AzAPI** covers newer/preview

resources; the **azuread** provider handles directory objects. Provider versions are **pinned**.

- **Remote state backend** — Terraform state (the record of what it manages) lives in an **Azure Storage

account **with a container and** blob lease locking**, versioned and access-controlled — never local.

- **Management groups** form a hierarchy under the Tenant Root Group; **Azure Policy** and **RBAC**

assigned at a management group **inherit** to every subscription and resource beneath it.

• **CI/CD** — Azure DevOps Pipelines (or GitHub Actions) authenticates to Azure with **workload identity federation (OIDC)**, runs** `plan` **on pull requests, and runs** `apply` **only after a** manual approval**.

What this solution builds (the components)

1. **Management group hierarchy + subscription vending**

• A hierarchy under the tenant root: `mg-corp-root` → `mg-platform` (`mg-identity`, `mg-management`, `mg-connectivity`), `mg-landingzones` (`mg-corp`, `mg-online`), `mg-sandbox`, `mg-decommissioned`.

• **Subscription vending** = a repeatable module that places a new subscription into the correct management group with its baseline policy, RBAC, and networking already inherited.

1. **Azure Policy guardrails as code**

• Assignments for **allowed regions**, **required tags**, and a **security baseline**, scoped at management groups. Deployed **audit/**`Disabled` **first**, promoted to `Deny` after validation.

1. **Identity & RBAC baseline**

• Least-privilege **role assignments** in code (platform admins, workload contributors, readers), targeting groups — not individuals.

1. **Hub-spoke networking & DNS**

• A **connectivity hub** VNet (with room for a firewall/gateway) and **spoke** VNets **peered** to it, plus **private DNS** zones — the network landing pad for workloads.

1. **Logging baseline**

• A central **Log Analytics workspace** and diagnostic-settings policy so activity is captured from the start.

1. **CI/CD pipeline with drift detection**

• `plan` on PR → **manual approval** → `apply`; a **scheduled** `plan` reports drift.

The standard deployment model

> ***Assess*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand*** *— inventory the current estate and decide standards (Assess); author the Terraform modules and pin versions (Configure);* `terraform plan`*/apply against a* ***non-production*** *management-group scope or sandbox first (Pilot); prove the gates — plan is clean, policy is in* ***audit****, state locks, pipeline approval works (Validate); then promote policy to* `Deny` *and vend real subscriptions/workloads in waves behind the approval gate (Expand). Every change is a reviewed PR and an approved pipeline run.*

What is out of scope

• **Workload** architecture inside the landing zones (app design, PaaS/IaaS specifics) — this builds the **foundation**, not the applications.

• The **decision** of allowed regions, the exact tag taxonomy, the RBAC model, and the **production change-approval **process — the guide** encodes** your standard but the standard is yours to set.

• Deep **Azure Firewall / ExpressRoute / VPN** design — the hub is provisioned with room for these; the detailed connectivity design is a separate networking engagement.

Authentication and connections used

• **Terraform** authenticates to Azure via the **Azure CLI** (local pilot) or **workload identity federation / OIDC** (pipeline) — no service-principal secrets in code.

• **State backend:** an Azure Storage account (`stiactftfstate*`) in a dedicated resource group, access controlled and versioned.

• **CI/CD:** Azure DevOps or GitHub, with a federated service connection scoped to the deployment target.

Limitations (be honest with buyers)

• **Version sensitivity:** Terraform, AzureRM/AzAPI/azuread providers, and any ALZ modules change frequently. All versions here are **pinned and flagged "verify"** — confirm against the registries.

• `apply` **changes real cloud resources.** Everything is **plan/what-if first**, against **non-prod** first, and **approval-gated** — never run an unreviewed apply against production.

• **Policy** `Deny` **can block legitimate deployments.** Guardrails start in **audit**; promote to `Deny` only after validating nothing legitimate is blocked.

• **Subscription creation** (EA/MCA billing) and **tenant-root** management-group changes need elevated rights and are governance-sensitive — treat them under change control.

2. Requirements Guide — Azure Landing Zone with Terraform

Verify **every** requirement before you plan or apply. Each entry: **Description · Why required · How to verify · How to implement if missing.** The category-wide prerequisites (a target subscription/tenant with rights, the IaC toolchain, a Git repo, a CI/CD platform + service connection, a remote state backend, secrets management, naming/tagging standards, and a non-production target + change control) are defined once in `global-requirements-devops.md` — confirm those first. The items below are **specific to building an Azure landing zone with Terraform**.

2.1 Elevated rights at the tenant-root / management-group scope

Description: The deploying identity can **create and manage management groups** and assign **policy and RBAC at the management-group scope** — typically **Owner** (or **User Access Administrator** + **Management Group Contributor**) elevated at the **root**. **Why required:** Landing-zone deployment creates the management-group hierarchy and assigns inherited policy/RBAC above the subscription level; subscription-only rights are not enough. **How to verify:** `az account management-group list` succeeds; the identity can create a test child management group. Run `IACTF-Discovery.ps1` (reports current hierarchy + your effective role). **How to implement if missing:** In Entra, enable **"Access management for Azure resources"** for the account (elevates a Global Admin to User Access Administrator at root), then assign the needed roles at the target management group. Remove the elevation when finished.

2.2 Terraform + provider versions pinned (and available)

Description: A **pinned** Terraform CLI version and pinned **AzureRM / AzAPI / azuread** provider versions (and any Azure landing-zone module versions) recorded in the repo. **Why required:** IaC is version-sensitive — an unpinned provider upgrade can change plan behavior or break a module mid-rollout. **How to verify:** `terraform version` and the `required_providers` block match the pinned versions; `terraform init` resolves them. *Verify current supported provider/module versions against the registry.* **How to implement if missing:** Pin `required_version` and each provider `version` in `main.tf` (examples are provided, flagged "verify"), commit the `.terraform.lock.hcl`, and record the versions.

2.3 Remote state backend provisioned (storage account + container + locking)

Description: An Azure **Storage account** (e.g. `stiactftfstate<unique>`) with a **container** (`tfstate`), **blob versioning**, and **least-privilege access** — the Terraform backend. **Why required:** Local state is unsafe and unshareable; remote state with blob-lease **locking** prevents two applies corrupting state. **How to verify:** The storage account/container exist and the deploy identity has **Storage Blob Data Contributor** on them. Run `IACTF-Validate.ps1` (checks backend reachability before plan). **How to implement if missing:** Create the resource group `rg-iactf-tfstate`, the storage account (with versioning + TLS1.2 + public-blob disabled), and the `tfstate` container; grant the deploy identity data access. (This one bootstrap step is done once, before the first `terraform init`.)

2.4 CI/CD service connection using workload identity federation (OIDC)

Description: A pipeline **service connection** to Azure that uses **workload identity federation (OIDC)** rather than a stored secret, scoped to the deployment target. **Why required:** Plan/apply must run through an **auditable pipeline** with a **secretless**, least-privilege identity — not from a laptop with a personal login. **How to verify:** A test pipeline authenticates to Azure via OIDC (no secret in variables). Run the pipeline's `plan` stage. **How to implement if missing:** In Azure DevOps/GitHub, create a **federated** service connection to the app registration / managed identity, assign it the needed role at the target management group, and reference it in `azure-pipelines.yml`.

2.5 Naming, tagging, and allowed-region standard defined

Description: The organization's **naming convention**, **required tags** (e.g. `owner`, `costCenter`, `environment`), and **allowed Azure regions**. **Why required:** These are encoded directly into the Terraform variables and the policy assignments (required-tags, allowed-regions); without a decided standard the guardrails are arbitrary. **How to verify:** A documented standard exists and maps to the `variables.tf` inputs and policy parameters. **How to implement if missing:** Decide and document the standard (this is a governance decision), then set it in the Terraform variables and policy parameters. *The standard itself is the customer's decision.*

2.6 A non-production target scope to validate against

Description: A **non-production** management-group scope (or a **sandbox** subscription) where `plan`/`apply` is proven before touching production. **Why required:** Every change is validated with plan/what-if against non-prod and approved before prod — this is the pilot target. **How to verify:** A `mg-sandbox` (or non-prod subscription) exists and the deploy identity can apply there. **How to implement if missing:** Create the sandbox management group / obtain a non-prod subscription and point the pilot Terraform workspace at it.

2.7 Microsoft Graph / Az PowerShell for the helper scripts

Description: The **Az PowerShell** module (and Azure CLI) on the operator's workstation for the discovery/validation/report helpers. **Why required:** The read-only helper scripts (`IACTF-Discovery.ps1`, `IACTF-Validate.ps1`, `IACTF-Report.ps1`) use Az/az to inventory the estate and check policy compliance around the Terraform run. **How to verify:** `Get-Module -ListAvailable Az.Accounts` returns a version and `Connect-AzAccount` succeeds; `az version` works. **How to implement if missing:** `Install-Module Az -Scope CurrentUser`; install the Azure CLI; sign in with `Connect-AzAccount` / `az login`.

Checklist

- Category global requirements confirmed (`global-requirements-devops.md`)
- Elevated rights at the management-group / root scope — `IACTF-Discovery.ps1`
- Terraform + provider/module versions **pinned** — *verify current versions*
- Remote state backend (storage account + `tfstate` container + locking) — `IACTF-Validate.ps1`
- CI/CD service connection via **workload identity federation (OIDC)**
- Naming / tagging / allowed-region standard defined and mapped to variables
- Non-production target scope (`mg-sandbox` / sandbox subscription) available
- Az PowerShell + Azure CLI installed and signed in

Global Requirements — DevOps (category-level)

These requirements apply to every solution in the DevOps category (Infrastructure-as-Code, pipelines, policy-as-code). They are published once at the category level so individual listings stay small. Confirm these before deploying any IaC/automation solution. Each entry: Description / Why / Verify /

Implement-if-missing.

> *Infrastructure-as-code changes real cloud resources. Everything runs plan/what-if first, through a reviewed pipeline, against a non-production target before production — never an unreviewed apply against prod. State and pipeline credentials are treated as secrets.*

G1. Target cloud subscription/tenant with sufficient rights

Description: An Azure subscription (and Entra tenant) where resources will be created, with Owner or an equivalent scoped role at the deployment scope. Why: Landing-zone and policy deployments create management groups, role assignments, and resources that need elevated rights. Verify: `az account show` succeeds; the identity holds Owner/User Access Administrator at the target scope. Implement: Obtain a subscription and the required role assignment (or a service principal / workload identity with those rights).

G2. IaC tooling installed and versioned

Description: The required toolchain — Terraform CLI, Bicep CLI / Azure CLI, and Git — at pinned versions. Why: IaC is version-sensitive; provider/module/CLI drift breaks reproducibility. Verify: `terraform version`, `az bicep version`, `az version`, and `git --version` return supported builds. *Verify current provider/module versions.* Implement: Install and pin the toolchain versions; record them in the repo.

G3. Source control repository

Description: A Git repository (Azure Repos or GitHub) to hold the IaC and pipeline definitions. Why: Code review, history, and CI/CD all depend on version control; IaC not in Git defeats the purpose. Verify: The repo exists and the operator can clone/push. Implement: Create the repository and set branch protection before authoring infrastructure code.

G4. CI/CD platform and service connection

Description: A pipeline platform (Azure DevOps Pipelines or GitHub Actions) with a secure connection to the cloud (workload identity federation preferred over secrets). Why: Plan/apply must run through an auditable pipeline with least-privilege, not from a laptop. Verify: A pipeline can authenticate to the subscription via OIDC/workload identity or a scoped service principal. Implement: Configure the service connection using workload identity federation; scope its rights to the deployment target.

G5. Remote state / deployment backend

Description: A secure remote backend for state (e.g. Azure Storage for Terraform state) or Azure deployment stacks for Bicep. Why: Local state is unsafe and unshareable; remote state with locking prevents corruption and drift. Verify: The state storage account/container (or deployment-stack scope) exists and is access-controlled. Implement: Provision the state backend with encryption, versioning, and least-privilege access before first apply.

G6. Secrets management

Description: A secrets store (Azure Key Vault) for any credentials the pipeline needs; no secrets in code. Why: Committed secrets leak; IaC repos are frequently shared and reviewed. Verify: No secrets in the repo (scanning enabled); pipeline pulls secrets from Key Vault / secure variables. Implement: Move all secrets to Key Vault / pipeline secret store; enable secret scanning on the repo.

G7. Naming, tagging, and region standards

Description: An organizational naming convention, required tags, and allowed regions. Why: Consistent naming/tagging is needed for governance, cost management, and policy enforcement. Verify: A documented standard exists and is encoded in the IaC modules/policy. Implement: Define the standard and encode it as policy-as-code and module defaults.

G8. Change-control and non-production target

Description: A non-production subscription/scope to validate against, and a change-approval process for production. Why: Every change is validated with plan/what-if against non-prod and approved before prod apply. Verify: A non-prod target exists; the pipeline requires approval before prod stages. Implement: Provision a non-prod scope and add manual approval gates to the production pipeline stages. *Production change approval is the customer's operational decision.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Azure Landing Zone with Terraform

Follow these steps **in order**. Work in your **Git repository** (the Terraform code), the **Azure CLI / portal** (bootstrap + verification), and the **CI/CD pipeline** (plan → approve → apply). Names in `code` are the exact objects to create — keep them consistent so the Terraform, pipeline, and helper scripts all refer to the same objects. Provider/module versions are **pinned and flagged "verify"** — confirm current supported versions before applying.

Object naming convention: `<purpose>-<scope>-<stage>` (Azure resource naming). The objects you build:

Object	Name
Root management group	`mg-corp-root`
Platform MGs	`mg-platform` → `mg-identity`, `mg-management`, `mg-connectivity`
Landing-zone MGs	`mg-landingzones` → `mg-corp`, `mg-online`
Sandbox / decommissioned	`mg-sandbox`, `mg-decommissioned`
State backend RG / storage	`rg-iactf-tfstate` / `stiactftfstate<unique>` (container `tfstate`)
Policy assignment (regions)	`IACTF-Policy-AllowedRegions-Root`
Policy assignment (tags)	`IACTF-Policy-RequireTags-LandingZones`

Object	Name
Log Analytics workspace	`log-iactf-management`
Hub network RG / VNet	`rg-iactf-hub` / `vnet-hub-eastus`
Spoke network RG / VNet	`rg-iactf-spoke-corp` / `vnet-spoke-corp-eastus`
CI/CD service connection	`sc-iactf-oidc`
Pipeline	`IACTF-Pipeline-Plan-Apply`

> ***Model:*** *Assess → Configure →* ***Pilot against*** `mg-sandbox` ***/ non-prod first*** *→ Validate the gates → Expand to production behind the approval gate. Everything is* `terraform plan` ***/ what-if first****. Every production* `apply` *is approval-gated. Rollback guidance is at the end.*

Step 1 — Assess: baseline the current estate

1. Run `IACTF-Discovery.ps1` — inventories the current management-group hierarchy, subscriptions,

existing policy assignments, and role assignments, and confirms your effective rights at root.

1. Record what already exists so the Terraform imports/avoids it (don't recreate the Tenant Root Group).

2. Confirm the requirements checklist (§2), especially **elevated rights at root** and the **standards**.

Step 2 — Bootstrap the remote state backend (one-time, outside the pipeline)

1. Create the state backend with the CLI (this must exist before `terraform init`):

```
az group create -n rg-iactf-tfstate -l eastus
az storage account create -n stiactftfstate<unique> -g rg-iactf-tfstate -l eastus \
    --sku Standard_LRS --min-tls-version TLS1_2 --allow-blob-public-access false
az storage account blob-service-properties update -n stiactftfstate<unique> \
    -g rg-iactf-tfstate --enable-versioning true
az storage container create -n tfstate --account-name stiactftfstate<unique> --auth-mode login
```

1. Grant the deploy identity **Storage Blob Data Contributor** on the storage account.

2. Run `IACTF-Validate.ps1 -BackendOnly` to confirm the backend is reachable.

Step 3 — Configure: author the Terraform and pin versions

1. In your repo, add the provided `main.tf` (management groups, policy assignments, RBAC baseline,

hub-spoke networking, Log Analytics, and the `backend "azurerm"` block pointing at Step 2).

1. Set your inputs in `variables.tf` / `terraform.tfvars`: allowed regions, required tags, the root

management-group display name, and network address spaces — from your **standard** (§2.5).

1. **Pin** `required_version` and each provider `version` in the `terraform/required_providers` block;

run `terraform init` and **commit** `.terraform.lock.hcl`. *Verify the pinned provider/module versions against the registry first.*

Step 4 — Pilot: plan and apply against the non-production scope

1. Point the workspace at the **non-prod** target (`mg-sandbox` or a sandbox subscription) via a variable.
2. Run `terraform plan -out tfplan` **locally or in the pipeline's plan stage** and **read the what-if**:

confirm it will create `mg-corp-root` and children, the **audit-mode** policy assignments, the RBAC baseline, `log-iactf-management`, and the hub/spoke VNets — and **nothing unexpected**.

1. Apply to the pilot scope: `terraform apply tfplan`.
2. **Keep policy in audit first:** ensure `IACTF-Policy-AllowedRegions-Root` and

`IACTF-Policy-RequireTags-LandingZones` deploy with effect `Audit`**/**`Disabled` at this stage — do **not** start in `Deny`.

Step 5 — Wire the CI/CD pipeline (plan → approve → apply)

1. Add the provided `azure-pipelines.yml` to the repo; create the pipeline `IACTF-Pipeline-Plan-Apply`.
2. Configure the **federated (OIDC) service connection** `sc-iactf-oidc` scoped to the target management

group (no stored secret).

1. The pipeline runs `terraform plan` on pull requests / the `plan` stage and publishes the plan as an

artifact; the `apply` stage is gated by a **manual approval / environment check**.

1. Add a **scheduled** trigger that runs `plan` (drift detection) and fails/reports if the plan is

non-empty (out-of-band drift).

Step 6 — Validate the pilot (the gates)

Run `IACTF-Validate.ps1` and `IACTF-Report.ps1` and confirm:

• **G1 Plan is clean & reviewed** — `terraform plan` produces the expected what-if and **no** unexpected

destroys; the plan was reviewed in a PR.

• **G2 State locks** — a second concurrent `apply` is **blocked** by the blob lease (state locking works).

• **G3 Policy inherits (in audit)** — `IACTF-Report.ps1` shows the policy assignments present at the MG

scope and reporting **compliance** (audit), inherited by child scopes.

• **G4 Pipeline gate** — the `apply` stage **waits for approval**; the scheduled `plan` reports **no

drift** on an unchanged environment.

Pass = all four gates green. **Fail** = fix before promoting to production (see §6).

Step 7 — Expand: promote to production in waves

1. **Promote guardrails to enforce:** change the policy assignment effects from `Audit` to `Deny`

(allowed-regions, required-tags) **one at a time**, re-plan, and apply — after confirming `IACTF-Report.ps1` shows no legitimate resources would be blocked.

1. **Vend production subscriptions:** use the subscription-vending pattern to place new subscriptions

into `mg-corp` / `mg-online` with baseline policy/RBAC/networking inherited.

1. **Add spokes:** peer new `vnet-spoke-*` VNets to `vnet-hub-eastus` for each workload landing zone.

2. Run every change as a **reviewed PR** → `plan` → **approval** → `apply`; re-run `IACTF-Report.ps1` after

each wave to capture the governance/compliance state.

> ***Rollback (any step):*** *Terraform is your rollback — **revert the commit** and* `apply` *the previous known-good state, or* `terraform plan/apply` *a targeted change to restore. For a bad **policy*** `Deny`*, set the assignment effect back to* `Audit` *and re-apply. **Never** hand-edit resources in the portal to "fix" drift — that widens the gap between state and reality; correct it in code and re-apply. Root management-group and subscription-vending changes are **high-impact** — do them under change control.*

Order of operations

```
Assess (Discovery) → bootstrap state backend → author + pin Terraform →
plan/apply to non-prod (policy in audit) → wire pipeline (plan→approve→apply
+ scheduled drift) → validate the 4 gates → promote policy to Deny + vend
prod subscriptions in waves
```

5. Testing & Validation — Azure Landing Zone with Terraform

Validate against the **non-production** scope (`mg-sandbox` / sandbox subscription) first. Each test has a **pass/fail** signal. Everything is **plan/what-if first** — do not promote to production or enforce `Deny` until all four gates pass. Reference scripts are named in each row.

Test matrix

#	Gate	What you do	Pass signal	Fail signal
1	**G1 Clean plan**	Run `terraform plan` (or the pipeline `plan` stage) against non-prod	What-if matches intent; **no unexpected destroys/changes**	Plan proposes surprise deletes / drift

#	Gate	What you do	Pass signal	Fail signal
2	**G1 Reviewed**	Open the plan as a PR; a second person reviews	Plan reviewed + approved before apply	Apply run without review
3	**G2 State lock**	Start two `apply`s against the same state	Second is **blocked** by the blob lease	Both proceed (no locking)
4	**G2 Backend**	Run `IACTF-Validate.ps1 -BackendOnly`	State storage account/container reachable + versioned	Backend missing / not reachable
5	**G3 Hierarchy**	After apply, `IACTF-Discovery.ps1`	`mg-corp-root` + children exist as designed	Hierarchy missing/incorrect
6	**G3 Policy (audit)**	`IACTF-Report.ps1`	`IACTF-Policy-*` assigned at MG scope, effect **Audit**, inherited	Policy absent / already in Deny
7	**G3 Tag/region compliance**	Create a test resource violating a tag/region rule	Audit policy **reports** it non-compliant	Not evaluated
8	**G4 Approval gate**	Run the pipeline to the `apply` stage	`apply` **waits for manual approval**	Apply runs unattended
9	**G4 Drift detection**	Run the scheduled `plan` on an unchanged env	Plan is **empty (no drift)**; a manual portal change is then **detected**	Drift not surfaced

Automated checks

• `IACTF-Discovery.ps1` — inventories the management-group hierarchy, subscriptions, policy assignments, and RBAC so you can confirm the built structure matches the design.

• `IACTF-Validate.ps1` — pre-flight and post-apply checks: backend reachability + locking, a `terraform plan` in **check/detailed-exitcode** mode (non-zero = pending changes/drift), and provider version pinning. **Read-only / plan-only — never applies.**

• `IACTF-Report.ps1` — governance report: policy **compliance state** per assignment and scope, and a drift summary (resources changed outside Terraform) for the audit trail.

• `azure-pipelines.yml` — the plan→approve→apply pipeline itself; its `plan` stage is the primary what-if gate, its scheduled run is the drift check.

Evidence to capture

- The reviewed `terraform plan` output (PR artifact) for the pilot apply.
- `IACTF-Discovery.ps1` output showing the management-group hierarchy.
- `IACTF-Report.ps1` compliance CSV (policy assignments + compliance state) — the governance evidence.
- A pipeline run screenshot showing the **manual approval** gate before `apply`.
- A scheduled-`plan` run showing **no drift**, plus one showing a **detected** out-of-band change.

Exit criteria (before Expand / before `Deny`)

- G1 — plan is clean, reviewed, and matches intent (no surprise destroys)
- G2 — remote state backend reachable **and** locking blocks concurrent applies
- G3 — hierarchy built; policy assigned in **Audit** and inheriting; violations reported
- G4 — pipeline `apply` waits for approval; scheduled `plan` detects drift
- `IACTF-Validate.ps1` and `IACTF-Report.ps1` run clean against non-prod

6. Troubleshooting — Azure Landing Zone with Terraform

Symptom → likely root cause → fix. Start with `IACTF-Discovery.ps1` (what exists + your rights) and `IACTF-Validate.ps1` (backend + plan/drift) — most failures are a state-backend problem, insufficient rights at the management-group scope, a version drift, or a policy `Deny` blocking a legitimate resource.

`terraform init` fails on the backend

- **Root cause:** the state storage account/container doesn't exist yet, the deploy identity lacks

Storage Blob Data Contributor, or the `backend "azurerm"` block points at the wrong account/container.

- **Fix:** confirm Step 2 bootstrap ran (RG `rg-iactf-tfstate`, storage `stiactftfstate<unique>`,

container `tfstate`); grant the identity blob-data access; verify the backend block values. Run `IACTF-Validate.ps1 -BackendOnly`.

"Error acquiring the state lock" / apply hangs on lock

- **Root cause:** another `apply` holds the blob lease, or a previous run crashed and left a stale lock.
- **Fix:** wait for the other run to finish (locking is working as intended). Only if you're certain no

run is active, break the lease on the state blob (or `terraform force-unlock <ID>`) — **never** during a live apply.

`plan`/`apply` fails with authorization / "does not have permission" at MG scope

- **Root cause:** the identity lacks **Owner / User Access Administrator + Management Group Contributor**

at the target management group (subscription-only rights aren't enough to create MGs or assign inherited policy/RBAC).

- **Fix:** assign the needed roles at the target MG (or elevate at root via **Access management for Azure resources** for the one-time hierarchy creation, then remove). `IACTF-Discovery.ps1` reports effective rights.

Provider/plugin version errors or an unexpected plan after `init -upgrade`

- **Root cause:** provider or module versions **floated** (not pinned / lock file not committed), changing plan behavior mid-rollout.

- **Fix:** pin `required_version` and each provider `version`, commit `.terraform.lock.hcl`, and run `terraform init` without `-upgrade` in CI. Upgrade versions deliberately in their own reviewed PR. *Verify supported versions against the registry.*

Policy assignment is blocking a legitimate deployment (`RequestDisallowedByPolicy`)

- **Root cause:** a guardrail was promoted to `Deny` too early, or its parameters (allowed regions / required tags) are stricter than intended.

- **Fix:** set the assignment effect back to `Audit`, re-apply, and confirm with `IACTF-Report.ps1` what would be blocked; correct the parameters, then re-promote to `Deny` once clean. Audit-before-Deny is the rule.

Policy shows "not started" / no compliance results

- **Root cause:** compliance evaluation hasn't run yet (it's periodic), or the assignment scope doesn't cover the resources.

- **Fix:** trigger an on-demand scan (`az policy state trigger-scan`) or wait for the evaluation cycle; confirm the assignment scope (`IACTF-Report.ps1`). Newly-assigned policy can take time to report.

Management group changes don't appear / hierarchy looks wrong

- **Root cause:** management-group operations are eventually consistent and can lag; or the Terraform created a parallel structure instead of using the existing root.

- **Fix:** re-run `IACTF-Discovery.ps1` after a short wait; ensure the code references the **existing** Tenant Root Group as parent rather than recreating it. Don't duplicate the root.

Pipeline can't authenticate to Azure (OIDC)

- **Root cause:** the federated **service connection** `sc-iactf-oidc` isn't configured for workload identity federation, the federated credential subject/issuer doesn't match the pipeline, or the connection lacks a role at the target scope.
- **Fix:** confirm the service connection uses **workload identity federation** (not a secret), the federated credential matches the pipeline's subject, and the identity has the needed role at the target MG. Re-run the `plan` stage.

The `apply` stage ran without approval

- **Root cause:** the pipeline environment/stage has **no approval check**, or the approval was applied to the wrong environment.
- **Fix:** add a **manual approval / environment check** to the `apply` stage's environment in `azure-pipelines.yml`; verify the `plan` and `apply` stages are separated with the gate between them.

VNet peering fails

- **Root cause: overlapping address spaces** between hub and spoke, or the peering was created only on one side.
- **Fix:** ensure hub/spoke address ranges don't overlap; create peering **both directions** (the module does this — confirm both `hub`→`spoke` and `spoke`→`hub` resources applied). Re-plan.

Scheduled `plan` reports drift you didn't make

- **Root cause:** someone changed a resource **in the portal** (out-of-band), or a provider default changed after a version bump.
- **Fix:** reconcile by **re-applying the code** (restores desired state) — don't "fix" it in the portal.

If a provider default changed, review the diff and update the code intentionally. This is drift detection doing its job.

First moves for any landing-zone issue

1. `IACTF-Discovery.ps1` — what exists, and do you have rights at the target MG scope?
2. `IACTF-Validate.ps1` — is the backend reachable/locked, and does `plan` show unexpected changes/drift?
3. `IACTF-Report.ps1` — are policy assignments present and in the expected effect (Audit vs Deny)?
4. Only then change configuration — `plan` **first**, review the what-if, and keep the prior commit as your rollback (revert + apply).

7. Best Practices — Azure Landing Zone with Terraform

Plan-first, always

• **Never** `apply` **without reading the** `plan`**.** The what-if is the safety mechanism — review it in a pull request, confirm no unexpected destroys, and only then apply. Use `-detailed-exitcode` in CI so a non-empty plan is a signal, not a surprise.

• **Non-prod before prod.** Prove every change against `mg-sandbox` / a sandbox subscription first; production `apply` is always behind a **manual approval** gate.

State discipline

• **Remote state, always locked.** Keep state in the `azurerm` backend with blob-lease locking and versioning; never local, never committed. One state per environment/scope to limit blast radius.

• **Least-privilege state access.** Only the deploy identity gets **Storage Blob Data Contributor** on the state account; state can contain sensitive values, so treat it like a secret store.

• **Never hand-edit in the portal to fix drift.** Correct the code and re-apply — portal edits widen the gap between state and reality and are exactly what drift detection is meant to catch.

Guardrails (policy) as code

• **Audit before Deny.** Assign guardrails (allowed-regions, required-tags, security baseline) in `Audit`**/**`Disabled` first, confirm nothing legitimate is flagged (`IACTF-Report.ps1`), then promote to `Deny` one assignment at a time. A premature `Deny` blocks real deployments.

• **Assign at the right scope.** Put broad guardrails high (root / platform) so they **inherit**; scope workload-specific policy to the landing-zone MG. Don't assign the same policy at multiple scopes.

• **Parameterize from the standard.** Allowed regions and required tags come from your governance standard via variables — keep the standard in one place, referenced by both policy and module defaults.

Hierarchy & subscription vending

• **CAF-aligned, but lean.** Keep the hierarchy shallow and purposeful (Platform / Landing Zones / Sandbox / Decommissioned); don't over-nest.

• **Vend subscriptions through code.** Place new subscriptions into the correct MG via the vending pattern so baseline policy, RBAC, and networking are inherited from day one — no click-ops onboarding.

- **Target groups, not people, for RBAC.** Assign roles to Entra groups; manage membership separately.

Keep role assignments least-privilege and in code.

Networking

- **Hub-spoke with peering.** Central connectivity in the hub; workloads in spokes peered to it. Reserve

address space deliberately; don't overlap ranges (peering will fail).

- **Private DNS centrally.** Manage private DNS zones in the connectivity scope and link spokes, so name

resolution is consistent.

Pipeline & drift

- **Secretless CI/CD.** Authenticate with **workload identity federation (OIDC)** — no service-principal

secrets in variables. Scope the service connection to the deployment target only.

- **Schedule drift detection.** A scheduled `plan` that fails on a non-empty result surfaces out-of-band

changes early; reconcile by re-applying the code, not by editing the portal.

- **One change, one PR, one approval.** Small, reviewed changes keep plans readable and rollbacks trivial.

Version safety

- **Pin and lock everything.** Pin Terraform, providers, and modules; commit `.terraform.lock.hcl`.

Upgrade versions **deliberately** in their own PR with a reviewed plan — never let CI float versions. *Verify current supported versions against the registry before upgrading.*

Change safety

- **Pilot-first, gated, reversible** — prove the four gates on non-prod before production.

- **Rollback = revert + apply.** Your Git history is the rollback path; revert the commit and apply the

prior known-good state. Set a bad `Deny` back to `Audit` and re-apply.

- **High-impact = change control.** Root management-group changes, subscription vending, and version

upgrades get their own change window and review.

Hygiene

- **No secrets in code.** Backend auth via CLI/OIDC; any runtime secrets come from **Key Vault**, never

the repo. Enable secret scanning on the repo.

Where to go next (adjacent catalog areas)

• **Azure Security Baseline & Governance** — deepen the policy/Defender-for-Cloud posture on the landing

zone this builds.

• **Entra ID Governance — Access Reviews & Entitlement Management** — govern who gets the RBAC roles the

landing zone defines.

• **Zero Trust — Conditional Access Baseline** — control the sign-in conditions for access to the cloud

control plane.

8. Deliverables Summary — Azure Landing Zone with Terraform

What you receive

A complete, self-guided package to deploy an **Azure landing zone as code with Terraform** — a Cloud Adoption Framework-aligned management-group hierarchy with subscription vending, Azure Policy guardrails as code, an identity/RBAC baseline with locked remote state, hub-spoke networking and DNS, and a CI/CD pipeline that runs **plan** → **approve** → **apply** with **drift detection**. Everything is **plan/what-if first, non-prod first, and approval-gated**, with no secrets in code.

Documentation

• **Executive Summary** — the problem (click-ops + drift), the solution (five landing-zone components),

the outcome, the boundary, and the version-pinning note.

• **Solution Overview** — how Terraform, remote state, management groups, policy inheritance, and OIDC

CI/CD fit together; the components built; scope and out-of-scope (workloads and standards decisions).

• **Requirements Guide** — landing-zone-specific prerequisites (elevated root rights, pinned

provider/module versions, the state backend, an OIDC service connection, naming/tag/region standards, a non-prod target, and the Az toolchain), each with Verify / Implement, on top of the category globals.

• **Implementation Guide** — ordered steps: assess → bootstrap state → author + pin Terraform → plan/apply

to non-prod (policy in audit) → wire the pipeline → validate → promote to Deny + vend prod, with **every object named** using the `<purpose>-<scope>-<stage>` convention.

• **Testing & Validation** — the four gates (clean/reviewed plan, state locking, policy inheritance in

audit, pipeline approval + drift detection) as a pass/fail matrix, with the evidence to capture.

• **Troubleshooting** — symptom → root-cause → fix for the common failures (init/backend, state lock,

provider drift, policy Deny blocking, MG rights, pipeline OIDC, peering, drift).

• **Best Practices** — plan-first, state discipline, audit-before-Deny, subscription vending, hub-spoke,

secretless CI/CD, version pinning, and where to go next.

Deployable artifacts — plan/what-if first, no secrets

Artifact	Purpose	Default mode
`IACTF-Discovery.ps1`	Inventory the management-group hierarchy, subscriptions, policy assignments, and RBAC; confirm root rights	Read-only
`main.tf`	The core Terraform: management groups, policy assignments (audit-first), RBAC baseline, Log Analytics, hub-spoke VNets + peering, and the `azurerm` remote-state backend — provider/module versions **pinned & flagged verify**	Declarative (plan before apply)
`IACTF-Validate.ps1`	Pre/post-apply checks: backend reachability + locking, `terraform plan` in detailed-exitcode (drift) mode, provider-pin check	Plan-only / read-only
`azure-pipelines.yml`	CI/CD: `plan` on PR → **manual approval** → `apply`, plus a scheduled `plan` for **drift detection**; OIDC (secretless) auth	Plan → approve → apply
`IACTF-Report.ps1`	Governance report: Azure Policy **compliance state** per assignment/scope + drift summary (resources changed outside Terraform)	Read-only

All PowerShell helpers are **read-only / plan-only**; the pipeline gates every `apply` behind a **manual approval**; authentication is via **CLI/OIDC** — **no credentials or secrets live in the artifacts**.

Outcomes

- A **governed, versioned** Azure foundation deployed from code — repeatable across subscriptions.
- **No drift** — remote state + a scheduled plan catch out-of-band changes.
- **Inherited guardrails** — policy and RBAC assigned once, inherited by every subscription below.
- **Auditable** — every change is a reviewed PR and an approved pipeline run, with a compliance report.
- A **pilot-first**, gated, reversible rollout (revert + apply) with a documented rollback.

Assumptions & boundaries

• Provider/module **versions are pinned and flagged "verify"** — confirm current versions before applying.

• The **state backend** is bootstrapped once (outside the pipeline) before the first `init`.

• **Standards** (naming, tags, regions, RBAC model, prod change-approval) are the customer's decisions,

encoded by the guide.

• **Workload** architecture inside landing zones and deep firewall/ExpressRoute design are out of scope.

*Delivered as a **Complete Solution Package (PDF + Word + scripts)** — the full documentation set plus all five deployable artifacts (Terraform, pipeline, and PowerShell helpers) in a single ZIP, yours to keep and reuse on every rollout.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

IACTF-Discovery.ps1

Purpose: Baselines what already exists so the Terraform imports/avoids it and you deploy into a known state: the management-group hierarchy, subscriptions and their MG placement, existing policy assignments, role assignments at management-group scope, and your effective rights at root. Read-only — makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IACTF-Discovery.ps1 -CsvFolder C:\Temp\iactf Requires the Az PowerShell module (Az.Accounts, Az.Resources) and `Connect-AzAccount`. EXPECTED RESULTS Console tree/tables for management groups, subscriptions, policy + role assignments, plus optional CSVs. Use it to confirm you have rights at the target scope and to plan imports. .NOTES White-label — DevOps / IaC. Read-only against Azure Resource Manager. No credentials stored. .EXAMPLE .\IACTF-Discovery.ps1 > [CmdletBinding()] param([string]$CsvFolder)

How to run: powershell -ExecutionPolicy Bypass -File .\IACTF-Discovery.ps1 -CsvFolder C:\Temp\iactf Requires the Az PowerShell module (Az.Accounts, Az.Resources) and `Connect-AzAccount`. EXPECTED RESULTS Console tree/tables for management groups, subscriptions, policy + role assignments, plus optional CSVs. Use it to confirm you have rights at the target scope and to plan imports. .NOTES White-label — DevOps / IaC. Read-only against Azure Resource Manager. No credentials stored. .EXAMPLE .\IACTF-Discovery.ps1 > [CmdletBinding()] param([string]$CsvFolder)

IACTF-Report.ps1

Purpose: Produces the governance/audit evidence for the landing zone: the Azure Policy COMPLIANCE state per assignment and scope (so you can confirm guardrails are in Audit and what they would flag before promoting to Deny), plus a drift summary from `terraform plan` (resources changed outside Terraform). Read-only — never applies. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IACTF-Report.ps1 -TfDir . -CsvFolder C:\Temp\iactf-gov Requires Az PowerShell (Az.Accounts, Az.Resources, Az.PolicyInsights) and, for drift, terraform. EXPECTED RESULTS Compliance table (assignment / scope / compliant vs non-compliant counts) + a drift line, plus CSVs. Keep the compliance CSV as governance evidence; a non-empty drift means reconcile in code. .NOTES White-label — DevOps / IaC. Read-only. No credentials stored. Reconcile drift by re-applying code, never by editing the portal (06-best-practices). .EXAMPLE .\IACTF-Report.ps1 -CsvFolder C:\Temp\iactf-gov > [CmdletBinding()] param([string]$TfDir = ".", [string]$CsvFolder = ".\iactf-gov")

How to run: powershell -ExecutionPolicy Bypass -File .\IACTF-Report.ps1 -TfDir . -CsvFolder C:\Temp\iactf-gov Requires Az PowerShell (Az.Accounts, Az.Resources, Az.PolicyInsights) and, for drift, terraform. EXPECTED RESULTS Compliance table (assignment / scope / compliant vs non-compliant counts) + a drift line, plus CSVs. Keep the compliance CSV as governance evidence; a non-empty drift means reconcile in code. .NOTES White-label — DevOps / IaC. Read-only. No credentials stored. Reconcile drift by re-applying code, never by editing the portal (06-best-practices). .EXAMPLE .\IACTF-Report.ps1 -CsvFolder C:\Temp\iactf-gov > [CmdletBinding()] param([string]$TfDir = ".", [string]$CsvFolder = ".\iactf-gov")

IACTF-Validate.ps1

Purpose: The safety pre-flight around the Terraform run. Verifies (1) the remote state backend is reachable and versioned, (2) `terraform plan` in detailed-exitcode mode — exit 2 = pending changes/drift, exit 0 = no changes, exit 1 = error, and (3) that providers are PINNED and the lock file is committed. It NEVER applies — it is a plan/what-if-only gate. HOW TO RUN Full check: .\IACTF-Validate.ps1 -TfDir . -StateRg rg-iactf-tfstate -StateSa stiactftfstate<unique> Backend only: .\IACTF-Validate.ps1 -BackendOnly -StateRg rg-iactf-tfstate -StateSa stiactftfstate<unique> EXPECTED RESULTS PASS/FAIL lines for backend, provider pinning, and the plan exit code. A clean run = ready to review the plan and (via the pipeline, approval-gated) apply. .NOTES White-label — DevOps / IaC. Plan/what-if only — this script never runs `terraform apply`. Requires terraform + Az on PATH. No credentials stored. .EXAMPLE .\IACTF-Validate.ps1 -BackendOnly -StateRg rg-iactf-tfstate -StateSa stiactftfstateabc123 > [CmdletBinding()] param([string]$TfDir = ".", [string]$StateRg, [string]$StateSa, [string]$StateContainer = "tfstate", [switch]$BackendOnly)

How to run: Full check: .\IACTF-Validate.ps1 -TfDir . -StateRg rg-iactf-tfstate -StateSa stiactftfstate<unique> Backend only: .\IACTF-Validate.ps1 -BackendOnly -StateRg rg-iactf-tfstate -StateSa stiactftfstate<unique> EXPECTED RESULTS PASS/FAIL lines for backend, provider pinning, and the plan exit code. A clean run = ready to review the plan and (via the pipeline, approval-gated) apply. .NOTES White-label — DevOps / IaC. Plan/what-if only — this script never runs `terraform apply`. Requires terraform + Az on PATH. No credentials stored. .EXAMPLE .\IACTF-Validate.ps1 -BackendOnly -StateRg rg-iactf-tfstate -StateSa stiactftfstateabc123 > [CmdletBinding()] param([string]$TfDir = ".", [string]$StateRg, [string]$StateSa, [string]$StateContainer = "tfstate", [switch]$BackendOnly)

azure-pipelines.yml

Purpose: Deploys the landing-zone Terraform through an auditable, plan-before-apply pipeline: Stage 1 Validate — fmt + init + validate + `terraform plan` (what-if), published as an artifact Stage 2 Apply — downloads the SAME plan and applies it, GATED by a manual approval Scheduled run — `terraform plan -detailed-exitcode` for DRIFT DETECTION (fails on drift) HOW IT WORKS / HOW TO RUN - PR / CI trigger runs Validate only (plan, no apply) so reviewers see the what-if in the PR. - Merge to main runs Validate then Apply; Apply waits on the 'prod-landingzone' Environment's manual-approval check (configure the approver in Pipelines > Environments). - A daily schedule runs the Drift stage; a non-empty plan (exit 2) fails the build to surface drift. - Auth is WORKLOAD IDENTITY FEDERATION (OIDC) via the service connection — NO stored secret. EXPECTED RESULTS Validate publishes 'tfplan'. Apply applies ONLY after approval. Drift run is green when state==code. NOTES White-label — DevOps / IaC. Plan-first, approval-gated. No secrets in this file. *** Pin the Terraform version and the service-connection name to your environment (flagged verify).** *
==========

How to run: - PR / CI trigger runs Validate only (plan, no apply) so reviewers see the what-if in the PR. - Merge to main runs Validate then Apply; Apply waits on the 'prod-landingzone' Environment's manual-approval check (configure the approver in Pipelines > Environments). - A daily schedule runs the Drift stage; a non-empty plan (exit 2) fails the build to surface drift. - Auth is WORKLOAD IDENTITY FEDERATION (OIDC) via the service connection — NO stored secret. EXPECTED RESULTS Validate publishes 'tfplan'. Apply applies ONLY after approval. Drift run is green when state==code. NOTES White-label — DevOps / IaC. Plan-first, approval-gated. No secrets in this file. *** Pin the Terraform version and the service-connection name to your environment (flagged verify).** *
==========

main.tf

Purpose: Deploys a Cloud Adoption Framework-aligned Azure landing zone as code: - Management group hierarchy (root -> platform / landing-zones / sandbox) - Azure Policy guardrails as code (allowed regions, required tags) — AUDIT FIRST - Identity / RBAC baseline (role assignment to a group, not a person) - Central Log Analytics workspace (logging baseline) - Hub-spoke networking with bidirectional peering + private DNS State is stored in a remote azurerm backend (bootstrapped once, see 03-implementation Step 2). HOW TO RUN (plan/what-if FIRST — never apply blind) terraform init terraform plan -out tfplan # review the what-if in a pull request terraform apply tfplan # pilot: target mg-sandbox / non-prod first EXPECTED RESULTS plan creates the MG hierarchy, AUDIT-mode policy assignments, RBAC, Log Analytics, and hub+spoke VNets — and NOTHING unexpected. Promote policy Audit->Deny only after validation (Step 7). NOTES White-label — DevOps / IaC. No secrets in code (auth via Azure CLI or pipeline OIDC). *** All provider/module versions below are PINNED and FLAGGED "verify" — confirm current supported versions against the Terraform Registry before you apply.** *